Neural Network Cart-Pole Controller from Scratch

Project Reference

Sebastien

June 2026

1 Overview

The goal of this project is to train a single multilayer perceptron (MLP) to swing up and balance an inverted pendulum on a cart, starting from the stable hanging position. The entire implementation is from scratch in Python and NumPy — no deep learning frameworks are used. The network, the backpropagation, the physics simulation, and the training loop are all hand-written.

The final approach is supervised imitation learning: a classical two-phase expert controller (energy-shaping swing-up followed by LQR balance) generates a dataset of state–action pairs, and the MLP is trained by ordinary regression to reproduce the expert's behaviour.

2 Project arc

The project went through several training paradigms before arriving at imitation learning.

The first approach was **backpropagation through time (BPTT)** through a differentiable Euler-integrated simulator. A two-stage curriculum was used: the first stage trained balance from near-upright with gradually increasing episode length, the second loaded those weights and progressively increased the starting angle toward the hanging position to learn swing-up. Training was plagued by instability — momentum runaway, limit cycles, catastrophic forgetting of balance during swing-up training — which led to increasingly elaborate stability machinery (spike detection, checkpoint restore, plateau detection, gated loss penalties).

After extensive tuning the best result was roughly one second of balance. Analysis of the gradient revealed a structural ceiling: the BPTT adjoint as implemented was an *open-loop* (truncated) gradient that omits the policy-feedback term. The true closed-loop gradient of the loss with respect to the weights includes a chain through the policy's dependence on state,

$$\left.\frac{\partial u_{t+1}}{\partial W}\right|_{\text{closed}} = \frac{\partial \pi(s_{t+1}; W)}{\partial W} + \frac{\partial \pi}{\partial s_{t+1}} \frac{\partial s_{t+1}}{\partial W},$$

but the implementation only captured the first (direct) term and propagated state sensitivities through the physics without feeding them back through the network. A finite-difference check confirmed a persistent $\sim$6% gradient mismatch attributable to this missing term. Without it, the gradient cannot represent the tight closed-loop coupling that stabilising feedback requires, so no amount of curriculum tuning could overcome the limitation.

A brief detour into **advantage actor–critic RL** also failed: the policy found a spinning local optimum rather than learning to balance.

This motivated a pivot to **Track B: supervised imitation learning**, which separates the control problem (solved by a hand-coded classical expert) from the function-approximation problem

(solved by standard backpropagation on independent state–action pairs). The remainder of this document describes the current implementation.

3 Dynamics

Consider a cart of mass M on a frictionless track at position x, with a pendulum of mass m and length l pinned to it. The angle θ is measured from the upward vertical, so $\theta = 0$ is the unstable upright equilibrium and $\theta = \pi$ is the stable hanging position. The bob position is

$$x_b = x + l\sin\theta, \qquad y_b = l\cos\theta.$$

The kinetic and potential energies are

$$\begin{aligned} T &= \tfrac{1}{2}(M+m)\dot{x}^2 + ml\dot{x}\dot{\theta}\cos\theta + \tfrac{1}{2}ml^2\dot{\theta}^2, \\ V &= mgl\cos\theta, \end{aligned}$$

and the Euler–Lagrange equations give

$$(M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F, \tag{1}$$

$$ml^2\ddot{\theta} + ml\ddot{x}\cos\theta - mgl\sin\theta = 0. \tag{2}$$

The simulator treats the network output as the cart acceleration $\ddot{x}$ directly and integrates the pendulum equation in the form

$$\ddot{\theta} = \frac{g}{l}\sin\theta - \frac{\ddot{x}}{l}\cos\theta, \tag{3}$$

using forward Euler at fixed timestep $dt = 0.027$, with $g = 10$ and $l = 1$. The function $\mathrm{wrap}(\theta) = ((\theta + \pi) \bmod 2\pi) - \pi$ maps angles into $(-\pi, \pi]$ relative to upright.

4 Expert controller

The expert is a two-phase controller: an energy-shaping swing-up pump that injects mechanical energy each half-swing, followed by a linear-quadratic regulator (LQR) that catches the pole and holds it. A threshold on $|\mathrm{wrap}(\theta)|$ and $|\dot{\theta}|$ selects between the two phases.

4.1 Energy-shaping swing-up

Define the pendulum's mechanical energy as

$$E = \tfrac{1}{2}l^2\dot{\theta}^2 + gl\cos\theta.$$

At the upright equilibrium $E_{\text{top}} = gl$. The time derivative along trajectories of (3) is $\dot{E} = -l\,\ddot{x}\,\dot{\theta}\cos\theta$, so $\ddot{x}$ has direct authority over the rate of energy injection. The swing-up control is

$$u_{\text{swing}} = k_E\,(E - E_{\text{top}})\,\mathrm{sign}(\dot{\theta}\cos\theta) \;-\; k_x\,x \;-\; k_{\dot{x}}\,\dot{x}, \tag{4}$$

where the first term is a proportional energy pump whose magnitude scales with the energy deficit, the sign function selects the pumping direction, and the k_x, $k_{\dot{x}}$ terms provide linear cart-centering to prevent runaway drift during the swing. The tuned gains are $k_E = 0.07$, $k_x = 0.2$, $k_{\dot{x}} = 0.3$.

4.2 LQR balance

Near $\theta = 0$, the dynamics (1)–(2) are approximately linear. Writing the state as $\boldsymbol{s} = (x, \dot{x}, \theta, \dot{\theta})^\top$ and the control as $u = \ddot{x}$, the linearisation gives

$$\dot{\boldsymbol{s}} = A\boldsymbol{s} + Bu,$$

with

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & g/l & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 \\ 1 \\ 0 \\ -1/l \end{pmatrix}.$$

The LQR cost is

$$J = \int_0^\infty \left(\boldsymbol{s}^\top Q\, \boldsymbol{s} + u^\top R\, u\right) dt,$$

with $Q = \mathrm{diag}(1, 1, 10, 1)$ and $R = 0.1$. The optimal linear feedback $u = -K\boldsymbol{s}$ is given by $K = R^{-1}B^\top P$, where P solves the continuous-time algebraic Riccati equation

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

P is computed once at module load via `scipy.linalg.solve_continuous_are`. At runtime the LQR action is simply $u = -K\boldsymbol{s}$, a dot product with the four-element gain vector K.

4.3 Phase switching

The expert switches from swing-up to LQR when both $|\mathrm{wrap}(\theta)| < 0.4$ and $|\dot{\theta}| < 3$. The thresholds must be set so the pump delivers the pole slowly enough that the linear approximation is valid when the LQR takes over.

5 Dataset generation

The expert is rolled out from many randomised initial conditions. For each of N_{roll} rollouts, the initial angle θ_0 is drawn uniformly from $(-\pi, \pi]$ and the initial angular velocity $\dot{\theta}_0$ from $[-v_{\max}, v_{\max}]$; the latter seeds fast near-top arrivals that teach the network how to catch. At each of the T_{roll} timesteps per rollout:

1. The expert's intended action $u^\star$ is recorded as the label.
2. The simulator is stepped with $u^\star + \epsilon$, where $\epsilon \sim \mathrm{Uniform}(-\sigma, \sigma)$ is exploration noise. This causes the trajectory to drift off the expert's ideal path, so the expert demonstrates recovery from states the network will actually encounter when it drives (addressing distribution shift).
3. Near-balanced states ($|\mathrm{wrap}(\theta)| < 0.4$ and $|\dot{\theta}| < 3$) are kept with probability p_{bal} to prevent the "hold still at the top" regime from flooding the dataset.

All states are stored with the angle wrapped to $(-\pi, \pi]$. The pairs are shuffled and saved as `.npz`:

$$X \in \mathbb{R}^{N\times 4}, \quad Y \in \mathbb{R}^N, \qquad X_i = \big(x,\ \dot{x},\ \mathrm{wrap}(\theta),\ \dot{\theta}\big)_i.$$

6 Network architecture

The network is a two-hidden-layer MLP with tanh activations and a linear output:

$$\begin{aligned} Z_1 &= W_1\boldsymbol{s} + b_1, & A_1 &= \tanh(Z_1), \\ Z_2 &= W_2A_1 + b_2, & A_2 &= \tanh(Z_2), \\ u &= w_3^\top A_2 + b_3, \end{aligned}$$

with $W_1 \in \mathbb{R}^{h\times 4}$, $W_2 \in \mathbb{R}^{h\times h}$, $w_3 \in \mathbb{R}^h$, and hidden width $h = 36$. Weights are initialised from $\mathcal{N}(0, 1/n_{\text{in}})$ (Xavier). The output u is the cart acceleration fed to (3).

7 Supervised training

7.1 Loss

The loss over a mini-batch of B state–action pairs is the mean squared error:

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\big(u(\boldsymbol{s}_i; W) - u_i^\star\big)^2.$$

7.2 Batched forward pass

States are stacked as rows of a matrix $X \in \mathbb{R}^{B\times 4}$. The batch convention transposes the single-example multiplications:

$$\begin{aligned} Z_1 &= X\,W_1^\top + b_1, & A_1 &= \tanh(Z_1), & &\in \mathbb{R}^{B\times h},\\ Z_2 &= A_1\,W_2^\top + b_2, & A_2 &= \tanh(Z_2), & &\in \mathbb{R}^{B\times h},\\ P &= A_2\,w_3 + b_3, & & & &\in \mathbb{R}^{B}. \end{aligned}$$

The cache (X, A_1, A_2) is saved for the backward pass. Pre-activations Z need not be stored because $\tanh'(z) = 1 - \tanh^2(z) = 1 - a^2$.

7.3 Batched backward pass

The MSE gradient at the output is

$$\delta P = \frac{2}{B}(P - Y) \;\in \mathbb{R}^B.$$

The $1/B$ factor comes from the mean in the loss. Propagating backward through each layer:

Output layer.

$$\begin{aligned} \nabla_{w_3} &= A_2^\top\,\delta P & &\in \mathbb{R}^h,\\ \nabla_{b_3} &= \textstyle\sum_i \delta P_i & &\in \mathbb{R},\\ \delta A_2 &= \delta P\,w_3^\top & &\in \mathbb{R}^{B\times h}. \end{aligned}$$

Hidden layer 2.

$$\begin{aligned} \delta Z_2 &= \delta A_2 \odot (1 - A_2^2) & &\in \mathbb{R}^{B\times h},\\ \nabla_{W_2} &= \delta Z_2^\top\,A_1 & &\in \mathbb{R}^{h\times h},\\ \nabla_{b_2} &= \textstyle\sum_i (\delta Z_2)_i & &\in \mathbb{R}^{h},\\ \delta A_1 &= \delta Z_2\,W_2 & &\in \mathbb{R}^{B\times h}. \end{aligned}$$

Hidden layer 1.

$$\begin{aligned} \delta Z_1 &= \delta A_1 \odot (1 - A_1^2) & &\in \mathbb{R}^{B\times h},\\ \nabla_{W_1} &= \delta Z_1^\top\,X & &\in \mathbb{R}^{h\times 4},\\ \nabla_{b_1} &= \textstyle\sum_i (\delta Z_1)_i & &\in \mathbb{R}^{h}. \end{aligned}$$

All six gradient arrays are ready for the weight update. The $\odot$ denotes elementwise (Hadamard) product; $\delta P\,w_3^\top$ is the outer product broadcasting each scalar error across the weight vector.

7.4 Training loop

The data is split into training (∼85%) and validation (∼15%) sets. Each epoch shuffles the training pairs and walks them in mini-batches:

1. Forward pass on the batch → predictions P.
2. MSE loss.
3. Backward pass → six gradient arrays.
4. Weight update: $W \leftarrow W - \eta \nabla_W$ (with optional momentum).

After each epoch, the validation MSE is computed via a single forward pass over the held-out set. Training stops when the validation loss flattens; continued descent in training loss beyond that point indicates memorisation rather than generalisation.

7.5 Deployment

A single-state wrapper `policy(state)` reshapes a state tuple into $[1, 4]$, calls the batched `forward`, and returns a scalar action. This is the function that replaces the expert inside the simulator loop. If input normalisation is used during training (subtracting the per-column mean and dividing by the standard deviation of the training set), the same transform must be applied in `policy` to keep training and deployment inputs consistent.

8 Lessons

The dominant lesson from the BPTT phase is that the truncated open-loop adjoint gradient imposes a hard ceiling on what stabilising feedback can be learned through a differentiable simulator. No amount of curriculum design or hyperparameter tuning can compensate for a structurally biased gradient. Recognising the difference between a tuning problem and an architectural limitation — and being willing to pivot — was the most important decision in the project.

The imitation learning approach reduced the training problem to its simplest possible form: stateless regression on independent pairs, with textbook two-layer backpropagation. The complexity moved from the training algorithm to the data generation, where careful coverage of the state space (varied starts, exploration noise, balanced-state downsampling) determines whether the trained network generalises to closed-loop deployment.